Clase 4: Módulos y Python Científico

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Contents

1. Módulos
2. Python científico
3. Scipy
4. Pandas

Módulos

Motivación

Ya conocemos módulos: `string`, `sys`

- `import string`
- `import sys`

Así se cargan ciertas funcionalidades.

Hoy aprenderemos a crear nuevos y conoceremos los más importantes en ciencia

Motivación

Ya conocemos módulos: `string`, `sys`

- `import string`
- `import sys`

Así se cargan ciertas funcionalidades.
Hoy aprenderemos a crear nuevos y conoceremos los más importantes en ciencia

Pip

Package **I**nstaller for **P**ython

- Gestor de paquetes que viene instalado junto a Python
- Info online en https://pypi.org/
- Fácil de usar por Bash

```
$ pip install pandas
$ python
> import pandas  # recién instalada!
```

Dónde están los programas...?

Pip

Package **I**nstaller for **P**ython

- Gestor de paquetes que viene instalado junto a Python
- Info online en https://pypi.org/
- Fácil de usar por Bash

```
$ pip install pandas
$ python
> import pandas  # recién instalada!
```

Dónde están los programas...?

el PATH

Cuando escribo `ls` en Bash... de dónde sale?

La variable de entorno PATH tiene las carpetas donde buscar programas[1]

Consola...(echo ${PATH})

[1] Se puede modificar para tener programas en nuevas carpetas, i.e. `/opt`

el PATH

Cuando escribo `ls` en Bash... de dónde sale?

La variable de entorno PATH tiene las carpetas donde buscar programas[1]

Consola...(echo ${PATH})

[1] Se puede modificar para tener programas en nuevas carpetas, i.e. `/opt`

Crear módulo

operaciones.py:

```
def suma(x,y): return x+y

def mult(x,y): return x*y
```

En Python:

```
> import operaciones # Debe estar en el path
> # from operaciones import suma,prod
> operaciones.suma(2,3) # 5
> operaciones.mult(2,3) # 6
```

el PATH en Python

```
~/clases/Bio210B$ python -m site
 sys.path = [
     '/home/nico/clases/Bio210B',
     '/usr/lib/python3/dist-packages' # son más...
```

Se puede extender con PYTHONPATH

```
$ export PYTHONPATH=/home/nico/otraCarpeta
$ python -m site
 sys.path = [
     '/home/nico/clases/Bio210B',
     '/home/nico/otraCarpeta',
     '/usr/lib/python3/dist-packages'
```

el PATH en Python

```
~/clases/Bio210B$ python -m site
 sys.path = [
     '/home/nico/clases/Bio210B',
     '/usr/lib/python3/dist-packages' # son más...
```

Se puede extender con PYTHONPATH

```
$ export PYTHONPATH=/home/nico/otraCarpeta
$ python -m site
 sys.path = [
     '/home/nico/clases/Bio210B',
     '/home/nico/otraCarpeta',
     '/usr/lib/python3/dist-packages'
```

Nota: variables en Bash

```
$ export PYTHONPATH=[...]
```

genera una variable de ambiente. Se puede ver con

```
$ echo ${PYTHONPATH}
```

Todas las definidas se ven con programa `env` de Bash

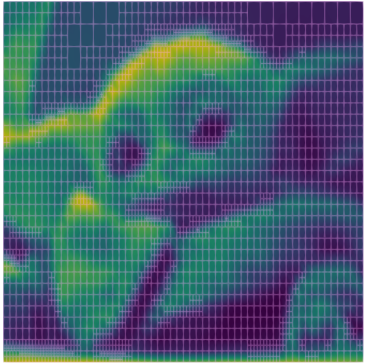
Contents

1. Módulos
2. Python científico
3. Scipy
4. Pandas

Python científico

Motivación

- Python es interpretado $\rightarrow$ lento
- Pero Python usa librerías *MUY rápidas*
- Eso + sintaxis lo vuelven ideal para la ciencia



math

```
> import math
> math. # Tab
math.acos(    math.copysign(  math.expm1(       math.lgamma(
math.acosh(   math.cos(       math.fabs(        math.log(
math.asin(    math.cosh(      math.factorial(   math.log10(
math.asinh(   math.degrees(   math.floor(       math.log1p(
math.atan(    math.dist(      math.fmod(        math.log2(
math.atan2(   math.e          math.frexp(       math.modf(
math.atanh(   math.erf(       math.fsum(        math.nan
math.ceil(    math.erfc(      math.gamma(       math.nextaft
math.comb(    math.exp(       math.gcd(         math.perm(
```

random

```
> import random
> random. # Tab
random.BPF            random.SystemRandom( random.gammavari
random.LOG4           random.TWOPI         random.gauss(
random.NV_MAGICCONST random.betavariate(  random.getrandbi
random.RECIP_BPF      random.choice(       random.getstate(
random.Random(        random.choices(      random.lognormva
random.SG_MAGICCONST random.expovariate(  random.normalvar
```

statistics

```
> import statistics as st
> st. # Tab
st.Counter(          st.bisect_left(  st.fabs(            s
st.Decimal(          st.bisect_right( st.fmean(           s
st.Fraction(         st.correlation(  st.fsum(            s
st.LinearRegression( st.covariance(   st.geometric_mean(  s
st.NormalDist(       st.erf(          st.groupby(         s
st.StatisticsError(  st.exp(          st.harmonic_mean(   s
```

Compatible con arreglos de numpy

NumPy

Numerical **Py**thon

- Permite hacer operaciones a grandes datos
- Operaciones *vectorizadas*
- Submódulos útiles:
  - numpy.fft (Fourier transform)
  - numpy.linalg (Linear algebra)
  - numpy.polynomial (Polinomios)
  - numpy.random (Generador de números)

Numpy arrays

```
> import numpy as np
> # 50 números equiespaciados en [0,1]
> num = np.linspace(0,1,50)
> # Números en [0,1) con paso 0.1
> num = np.arange(0,1,0.1)
```

Veamos tiempos de ejecución en sumar

Numpy arrays

```
> import numpy as np
> # 50 números equiespaciados en [0,1]
> num = np.linspace(0,1,50)
> # Números en [0,1) con paso 0.1
> num = np.arange(0,1,0.1)
```

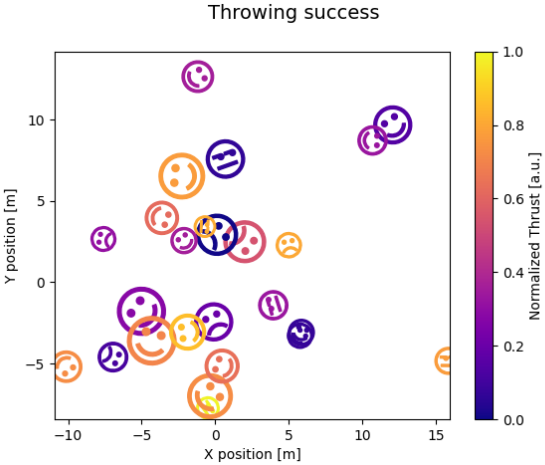
Veamos tiempos de ejecución en sumar

Operaciones vectorizadas

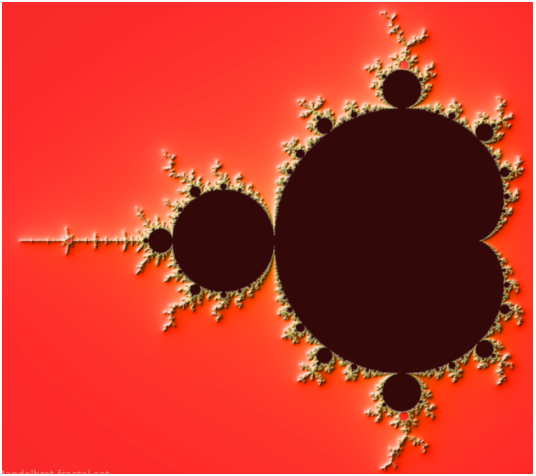
$$\text{vector:} \qquad x \in \mathbb{R}^N \rightarrow \left.\begin{bmatrix} 1 \\ 1.1 \\ \vdots \\ -0.1 \end{bmatrix}\right\} \text{N números}$$

```
> import numpy as np
> num = np.linspace(0,1,50)
> np.abs(num) # Val absoluto de todo
> np.exp(num) # exponencial
> np.sin(num) # seno, ... etc
> num.dot(num) # normas, producto interno, etc
```

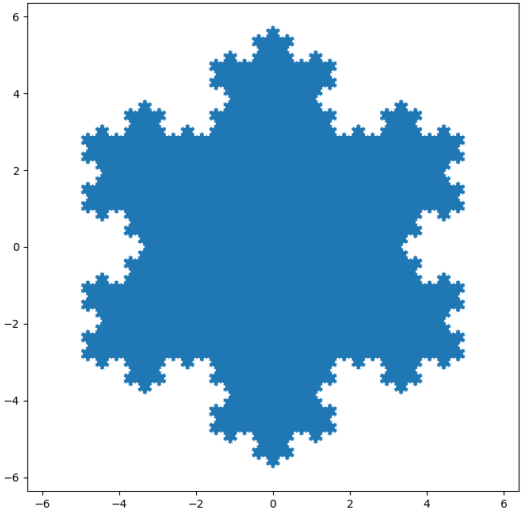
Matplotlib


Throwing success
Y position [m]
X position [m]
Normalized Thrust [a.u.]
10
5
0
−5
−10
−5
0
5
10
15
1.0
0.8
0.6
0.4
0.2
0.0

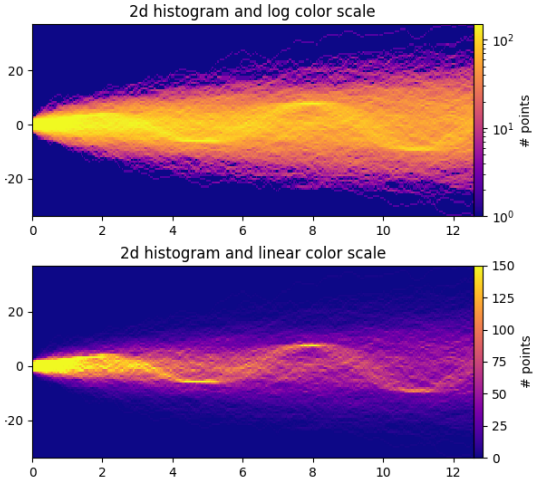
Matplotlib



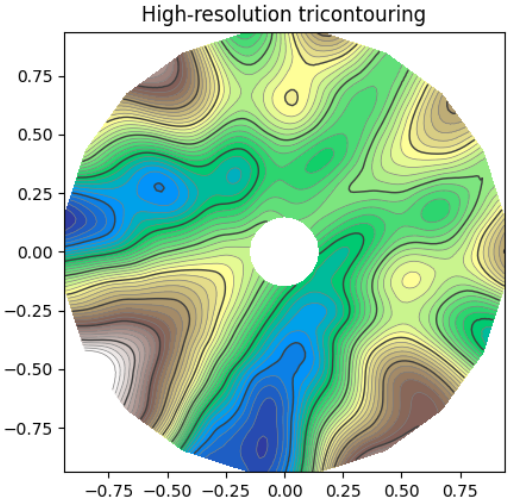
Matplotlib



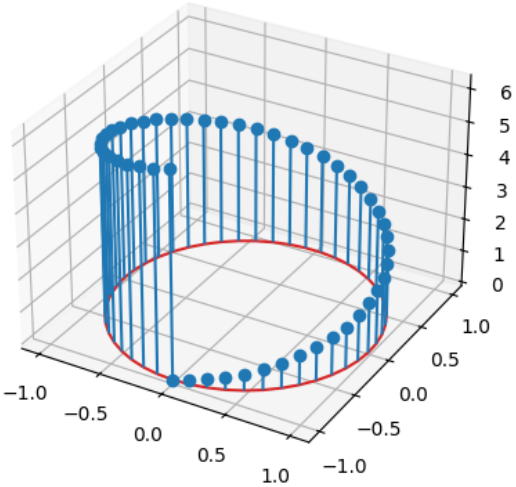
Matplotlib


2d histogram and log color scale
20
0
-20
0
2
4
6
8
10
12
10^2
10^1
10^0
points
2d histogram and linear color scale
20
0
-20
0
2
4
6
8
10
12
150
125
100
75
50
25
0
points

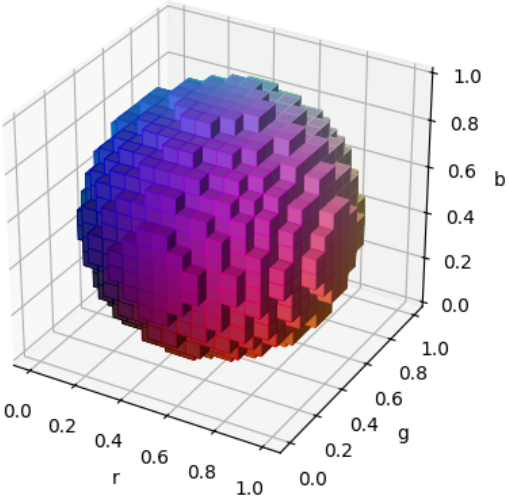
Matplotlib


High-resolution tricontouring
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75

Matplotlib



Matplotlib



Ejemplo simple

```
> import numpy as np
> import matplotlib.pyplot as plt
> xs = np.linspace(0, 2*np.pi, 1000) # [0,2 pi]
> ys = np.sin(xs)
> plt.plot(xs,ys) # Lista de puntos x e y
> plt.show()
```

Se puede agregar de todo...

Ver documentación online: `matplotlib.org`.

Ejemplo simple

```
> import numpy as np
> import matplotlib.pyplot as plt
> xs = np.linspace(0, 2*np.pi, 1000) # [0,2 pi]
> ys = np.sin(xs)
> plt.plot(xs,ys) # Lista de puntos x e y
> plt.show()
```

Se puede agregar de todo...
Ver documentación online: `matplotlib.org`.

Recap

- Dónde están los módulos por defecto
- Cómo agregar un módulo
- Cómo hacer módulo detectable con PYTHONPATH
- `math`
- `Numpy`
- `Matplotlib`

Clase 4: Módulos y Python Científico

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Mini ejercicios

operaciones.py

```
def suma(x,y): return x+y

def mult(x,y): return x*y
```

- Crear el módulo operaciones e importarlo desde un script.
- Guardar el módulo en otro lado e importarlo usando PYTHONPATH

Temas

- Scipy
- Pandas

Todas instalables con `pip`

Contents

1. Módulos
2. Python científico
3. Scipy
4. Pandas

Scipy

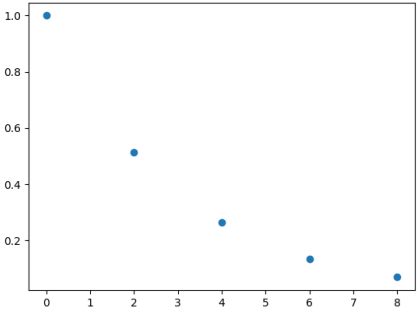
Scipy

- Librería científica
- Tiene *muchas* operaciones matemáticas
- Álgebra lineal (más eficiente que Numpy)
- Interpolación de datos
- Solución de ecuaciones
- Minimización de funciones
- Y MUCHO más

Scipy

- Librería científica
- Tiene *muchas* operaciones matemáticas
- Álgebra lineal (más eficiente que Numpy)
- **Interpolación de datos**
- **Solución de ecuaciones**
- **Minimización de funciones**
- Y MUCHO más

Scipy: Interpolación de datos

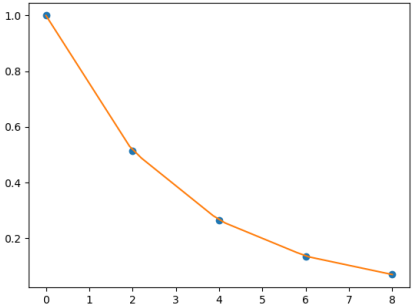


Y si quiero el valor en $x = 3$? Eso se llama *interpolación*[2]

[2] Extrapolación sería bajo 0 o sobre 8

Scipy: Interpolación de datos

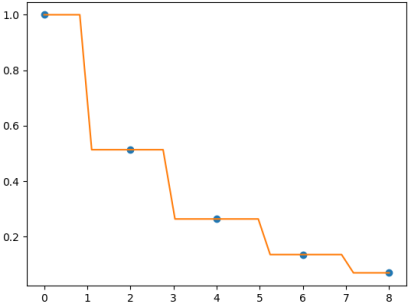
```
> import numpy as np
> import matplotlib.pyplot as plt
> from scipy import interpolate
> x = np.linspace(0,8,5)
> y = np.exp(-x/3)
> f = interpolate.interp1d(x, y, kind)
> xnew = np.linspace(0,8,100)
> ynew = f(xnew)
```



kind='linear'

Scipy: Interpolación de datos

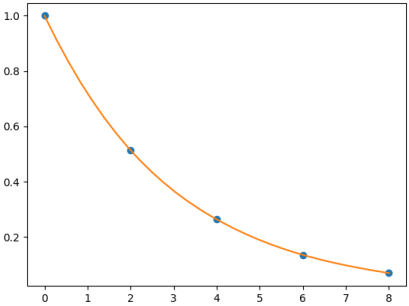
```
> import numpy as np
> import matplotlib.pyplot as plt
> from scipy import interpolate
> x = np.linspace(0,8,5)
> y = np.exp(-x/3)
> f = interpolate.interp1d(x, y, kind)
> xnew = np.linspace(0,8,100)
> ynew = f(xnew)
```



kind='nearest'

Scipy: Interpolación de datos

```
> import numpy as np
> import matplotlib.pyplot as plt
> from scipy import interpolate
> x = np.linspace(0,8,5)
> y = np.exp(-x/3)
> f = interpolate.interp1d(x, y, kind)
> xnew = np.linspace(0,8,100)
> ynew = f(xnew)
```

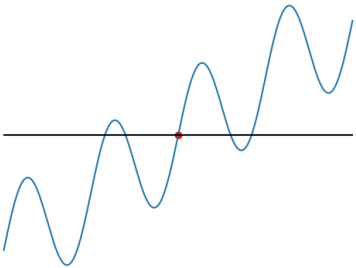


kind='cubic'

Scipy: Solución de ecuaciones ($f(x) = 0$)

$$f(x) := x + \sin x = 0$$

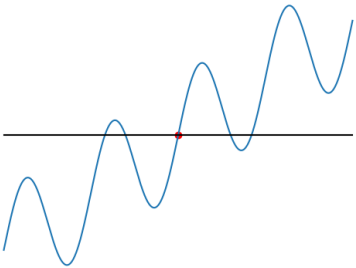
```
from scipy.optimize import root_scalar
def f(x): return x+sin(2*pi*x)
domx = np.linspace(-2,2,1000)
domy = [f(x) for x in domx]
x0,x1 = 3, -3
sol = root_scalar(f, x0=x0, x1=x1)
plt.scatter(sol.root, f(sol.root))
plt.plot(domx, domy);plt.show()
```



Scipy: Solución de ecuaciones ($f(x) = 0$)

$$f(x) := x + \sin x = 0$$

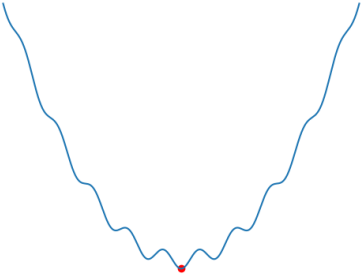
```
from scipy.optimize import root_scalar
def f(x): return x+sin(2*pi*x)
domx = np.linspace(-2,2,1000)
domy = [f(x) for x in domx]
x0,x1 = 3, -3
sol = root_scalar(f, x0=x0, x1=x1)
plt.scatter(sol.root, f(sol.root))
plt.plot(domx, domy);plt.show()
```



Scipy: Minimización

$$\min_x f(x) := \sin^2 4x + x^2 = 0$$

```
from scipy.optimize import minimize_scalar
def f(x): return sin(4*x)**2 + x**2
domx = np.linspace(-4,4,1000)
domy = [f(x) for x in domx]
sol = minimize_scalar(f)
plt.scatter(sol.x, f(sol.x))
plt.plot(domx, domy);plt.show()
```



Soluciones numéricas NO SON exactas

Contents

1. Módulos
2. Python científico
3. Scipy
4. Pandas

Pandas

Tipo central: DataFrame

```
import pandas
mydataset = {
  'cars': ["BMW", "Volvo", "Ford"],
  'passings': [3, 7, 2]
}
myvar = pandas.DataFrame(mydataset)
print(myvar)
$      cars    passings
  0     BMW          3
  1   Volvo          7
  2    Ford          2
```

DataFrame

- Es el elemento central de Pandas
- Son *tablas de datos*
- Se pueden filtrar fácilmente

Nombre de datos

```
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45] }

df = pd.DataFrame(data)
print(df)
df = pd.DataFrame(data, index = ["day1", "day2", "day3"]
print(df)
```

Cargar datos

Asumiendo que existe archivo `data.csv`

```
import pandas as pd

df = pd.read_csv('data.csv')
print(df)
```

Explorando datos

```
df = pd.read_csv('data.csv')
```

- `df.head(n)`: Primeros `n` datos
- `df.tail(n)`: Últimos `n` datos
- `df.describe()`: Ver estadísticas descriptivas
- `df.sort_values(by="A")`: Ordenar según columna "A"

Consola

Acceso a datos

```
df = pd.read_csv('data.csv')
```

- `df["A"]`: Extraer columna con etiqueta "A"
- `df[1:4]`: Extraer filas en [1,4)
- `df[1:4][["A", "B"]]`: Filas y columnas

Consola

Filtros booleanos

Exploremos los siguientes comandos:

```
df = pd.read_csv('data.csv')
df["Pulse"] > 120
df[df["Pulse"] > 120]
```

Se llama filtro por máscara, o máscara booleana.

Filtros booleanos

Exploremos los siguientes comandos:

```
df = pd.read_csv('data.csv')
df["Pulse"] > 120
df[df["Pulse"] > 120]
```

Se llama filtro por máscara, o máscara booleana.

Recap

- Python tiene muchas librerías para hacer ciencia
- Algunas simples: `math`, `statistics`, `random`
- Otras complejas: `scipy`, `numpy`
- Scipy: interpolar, minimizar, resolver
- Pandas sirve para procesar datos
- También se puede hacer estadística (no lo vimos)
- Máscaras booleanas

https://pandas.pydata.org/docs/user_guide/10min.html

Recap

- Python tiene muchas librerías para hacer ciencia
- Algunas simples: `math`, `statistics`, `random`
- Otras complejas: `scipy`, `numpy`
- Scipy: interpolar, minimizar, resolver
- Pandas sirve para procesar datos
- También se puede hacer estadística (no lo vimos)
- Máscaras booleanas

https://pandas.pydata.org/docs/user_guide/10min.html

Clase 4: Módulos y Python Científico

Nicolás Alejandro Barnafi Wittwer
Pontificia Universidad Católica de Chile
Centro de Modelamiento Matemático

Pontificia Universidad Católica de Chile

Mini ejercicios

- Genere una sucesión de 0's y 1's que representan lanzamientos de monedas. Calcule indicadores estadísticos de sus datos usando `statistics` y `pandas`
- Considere la función $f(x) = \sin x$ y encuentre numéricamente un mínimo con `scipy.minimize`. Qué pasa con los otros mínimos?